

Principal Component Analysis (PCA)

1) What is PCA?

PCA is a dimensionality reduction technique in machine learning.

- It helps reduce the number of features (columns) in your dataset while keeping most of the important information (variance).
- Benefits of PCA:-

1. faster training: - fewer features $\rightarrow$ less computation
 2. faster inference: - model predicts quicker
 3. Easier visualization: - reduce 100s of features to 2D or 3D for plotting
- Handles curse of dimensionality - reduces model complexity & overfitting.

2) Why do we need PCA?

- Real life datasets often have hundreds or thousands of features.
- Not all features impact the target variable equally:
 - Example: - predicting house prices
 - Important features:- area, price, etc.

impact more

impact less

Date _____

Page _____

	area	bathroom	plot	trees nearby	price
	2600	2	8500	2 less	550000
	3000	3	9200	2	565000
	3200	3	8750	2	610000
	3600	4	10200	2	680000
	4000	4	15000	2	725000
door	2600	2	7000	2	585000
door	2800	3	9000	2	615000
door	3300	4	10000	1	650000
door	3600	4	10500	1	710000

- less important features:- number of bathrooms
- Almost irrelevant features:- number of bees nearby
- PCA identifies the most important features, called principal components (PCs)

3. How PCA works (Conceptually)

Example 1:- Digits dataset

- Handwritten digits are represented as 8x8 pixel images $\rightarrow$ 64 features
- Some pixels are always black (not important) $\rightarrow$ can be ignored
- PCA identifies which pixels (features) contain most variance.

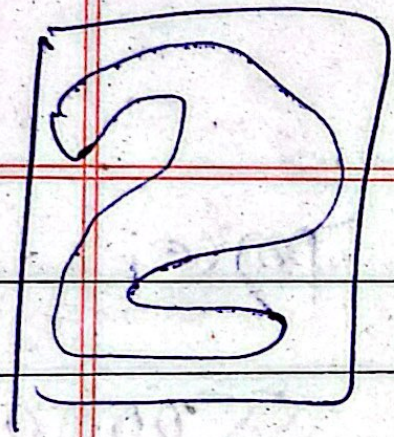
Example 2:- Iris flower dataset

- Scatter plot of sepal width vs sepal height.
- Draw a line along the maximum variance $\rightarrow$ principal component 1 (PC1)

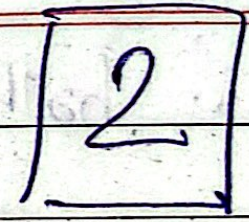
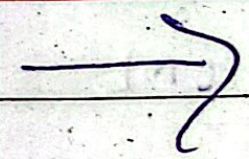
8x8 grid

Corner pixels

Date _____
Page _____

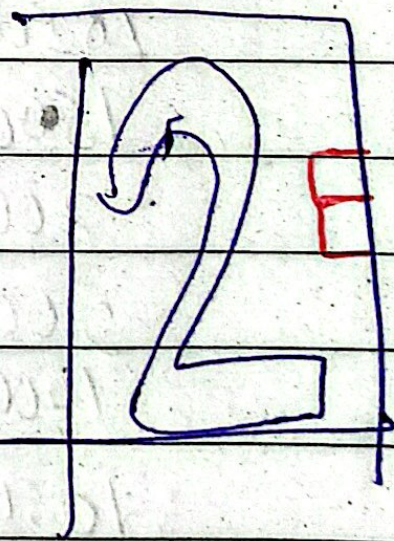
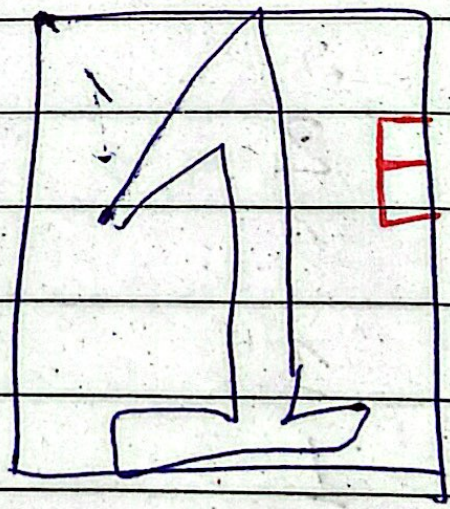
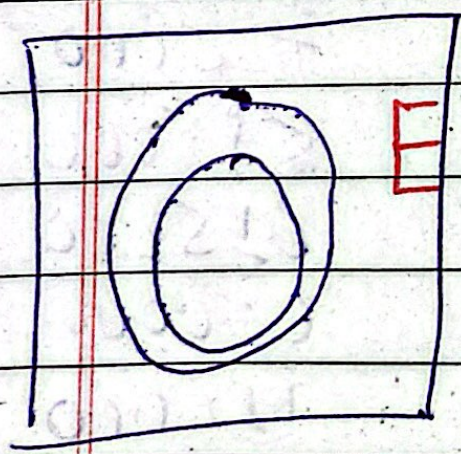


0	0	4	4	9	0	0	0
0	0	13	11	12	0	0	0
0	0	5	0	2	7	0	0
0	0	3	0	4	5	0	0
0	0	0	6	13	4	0	0
0	0	2	3	16	7	0	0
0	0	8	1	2	10	0	0
0	0	2	8	8	8	4	1



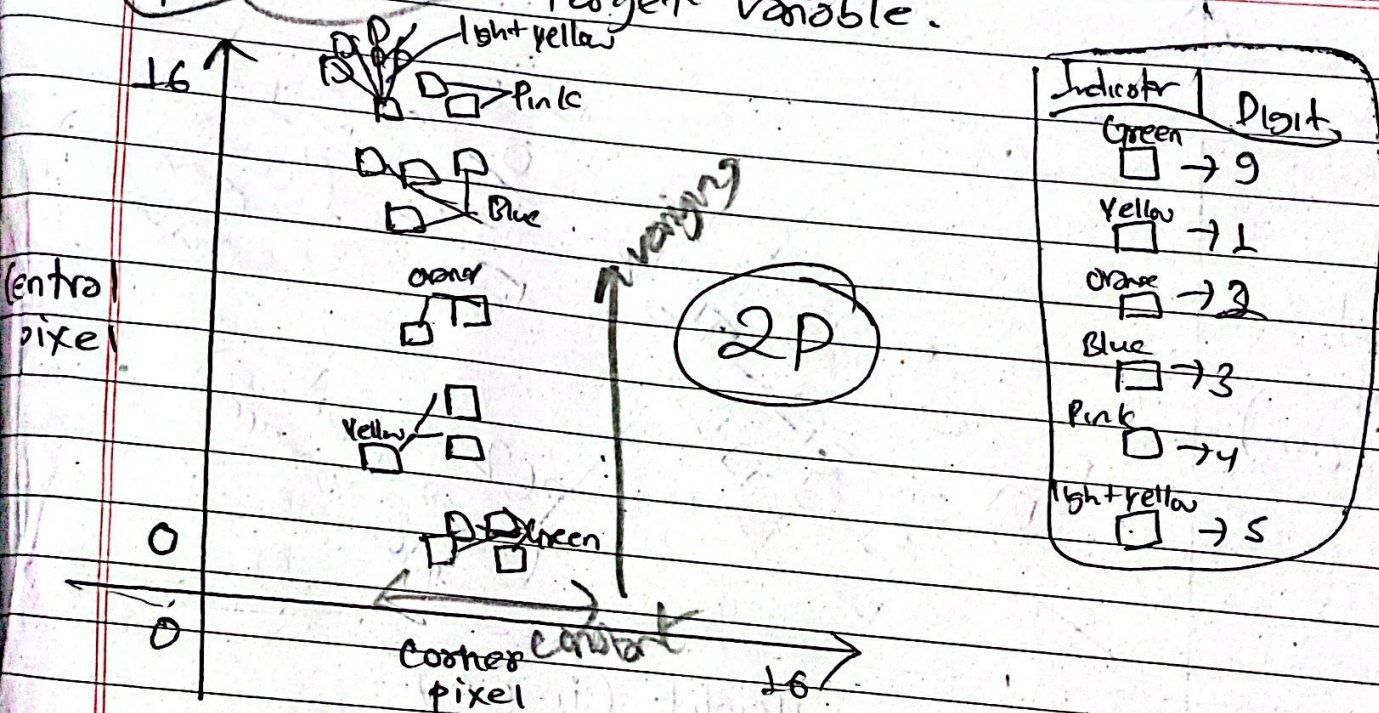
64 features

Ex:



No, matter what the numbers these pixels are black
are not important.

PCA is a process of figuring out most important features or principal components that has the most impact on the target variable.

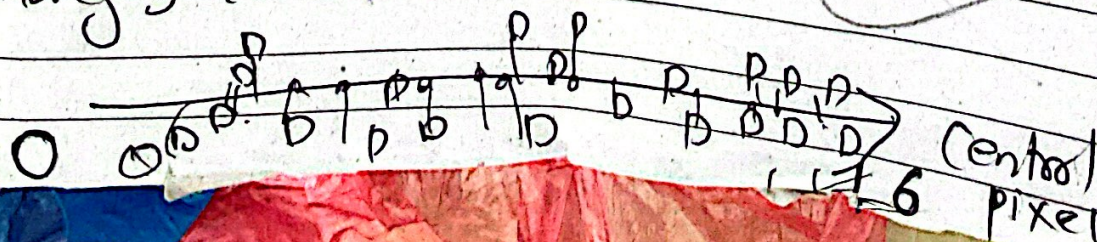


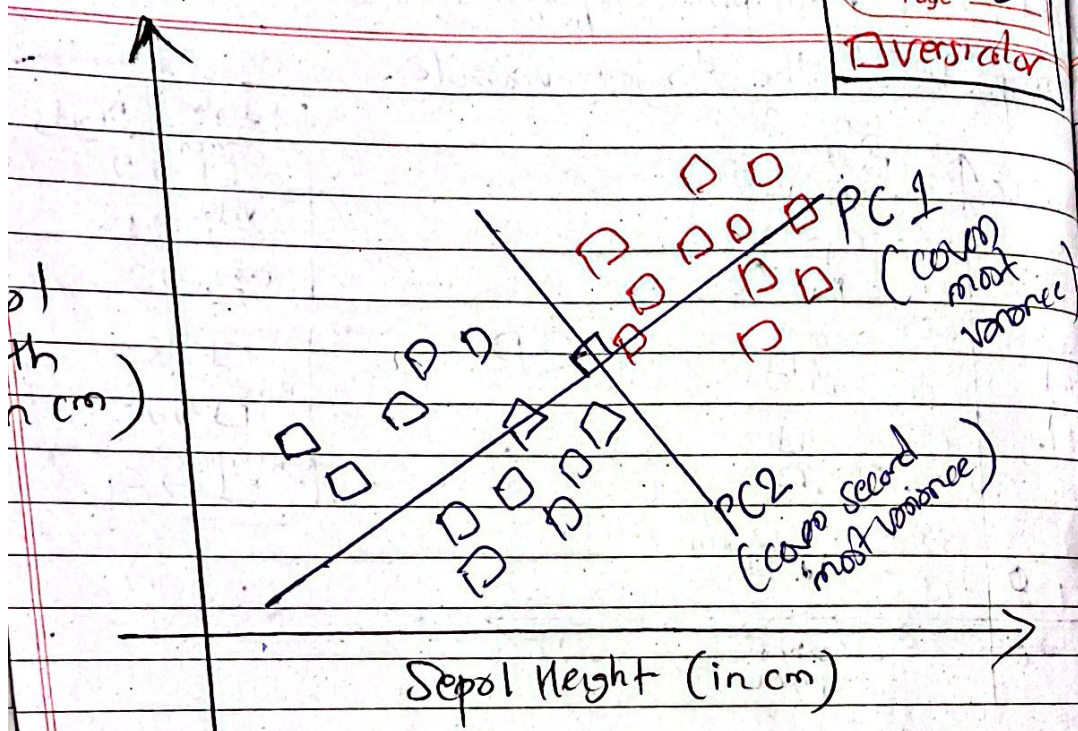
Every pixel value ranges from 0 to 16.
 0 = completely black
 16 = completely white

So, from the graph, we can clearly see corner pixel are almost constant (x-axis), while central pixel value are changing (y-axis).

So, central pixel is more important & corner pixel are not that important.

So changing to 1D.





(learn this later in Campus X)
Math is really complex here

- Perpendicular line $\rightarrow$ principal component 2 (PC2)
- PCA orders components by variance explained

key points:-

- PC1 $\rightarrow$ captures most variance
- PC2 $\rightarrow$ second most variance
- if there are 100 features, PCA can create 100 components, sorted by importance.

Important considerations before PCA

feature scaling is necessary:-

- If features are on different scales (eg. million vs 1-10), PCA may give skewed results.
- Use Standard Scaler or Min Max scaler

Trade-off

- Reducing too many features $\rightarrow$ lose information $\rightarrow$ lower accuracy
- Reducing too few $\rightarrow$ may not simplify computation

5. PCA in python (Using scikit-learn)

Date _____
Page _____

Step 1: - Import libraries

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
```

Step 2: - Load dataset

digits.keys()

```
digits = load_digits()
```

```
X = digits.data
```

```
y = digits.target
```

features (64 pixels)
Target (digit 0-9)

Step 3: - Scale features

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Step 4: - Split data

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.2, random_state=42
```


Step 5 - Train baseline model

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
print("Accuracy without PCA", accuracy)
```

~ 92%

Step 6 - Apply PCA

Retain 95% of variance

```
pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X_scaled)
print("Original shape:", X_scaled.shape)
print("PCA shape:", X_pca.shape)
```

(samples, 64)
(samples, 29)

Step 7 - Explained variance

```
print(pca.explained_variance_ratio_)
```

// shows how much information each PC captures

J.
Principal
Component

Step 8: - Train model with PCA data

Date _____
Page _____

```
X_train_pca, X_test_pca, y_train, y_test  
= train_test_split(  
    X_pca, y, test_size=0.2, random_state=42  
)
```

```
model.fit(X_train_pca, y_train)
```

```
accuracy_pca = model.score(X_test_pca, y_test)
```

```
print("Accuracy with PCA:", accuracy_pca)  
# 1.597
```

5. Using PCA with fixed number of components

• Example [n_components=2]

```
pca = PCA(n_components=2)
```

```
X_pca2 = pca.fit_transform(X_scaled)
```

```
print("Shape:", X_pca2.shape) # only 2 features
```

```
print(pca.explained_variance_ratio_)  
# usually lower, accuracy drops
```

• Trade-off: - fewer components $\rightarrow$ faster computation but lower accuracy

7. Key Takeaways

• PCA reduces high-dimensional data to fewer dimensions.

• Helps in:

• faster computation

• easier visualization

• avoiding overfitting

Imp: • Always scale features before PCA.

• Choose number of components based on variance or trial-and-error.

• Accuracy usually remains high if enough variance is retained.

Variance In PCA

- In PCA, variance refers how spread out the data is along each feature.
- features (columns) with high variance contain more information because they differ a lot between data points.
 - features with low variance or almost constant across data points $\rightarrow$ they don't add much information.

Example: -

- Predicting house prices
 - Area ranges from 500 to 5000 sq ft $\rightarrow$ high variance $\rightarrow$ important features
 - Number of trees nearby is 2 or 3 for most all houses $\rightarrow$ low variance $\rightarrow$ not very important

Why PCA uses variance: -

- PCA identifies directions (principal components) in which the data varies the most.
- By keeping components with high variance, PCA retains most of the important information & ignores unimportant features.

In short:-

- Variance in PCA = amount of information / spread in the data.
- PCA keeps the components / features with maximum variance because they capture + essence of the dataset.

Outlier is high variance

- Outliers are datapoints that are far away from the rest of the data.
- Outliers can artificially increase variance, making PCA think a feature is important when most of the data doesn't vary much.
- This can distort the principal components & affect results.

Example:- dataset of house prices mostly bet \$50k - \$500k.
• One house costs \$1 million + huge outlier.
• PCA sees that feature (price) has very high variance because of the outlier.
• But in reality, most data points don't vary much $\rightarrow$ PCA might overemphasize that outlier.

Soft/Bot problem:-

- Scale features (Standard or MinMax scaler) reduces impact of large numbers.
- Detect & remove outliers before PCA.

$\Rightarrow$ Variance is good to measure information, but extreme variance (outliers) always check data distribution before PCA.